SOFTWARE REQUIREMENTS SPECIFICATION

Calibration & Maintenance Computerized Management Information System (CMCMIS)

Prepared for: Space Applications Centre (SAC), Indian Space Research Organisation (ISRO)

Document Type: Software Requirements Specification (IEEE 830 / ISO-29148 style)

Version 1.0

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes, in full technical depth, the functional and non-functional requirements implemented by the CMCMIS (Calibration & Maintenance Computerized Management Information System) software developed for the Space Applications Centre (SAC), ISRO. The document has been compiled by direct, exhaustive inspection of the delivered source-code baseline — the React/Vite frontend ("FE"), the Node.js/Express backend ("BE"), the MySQL migration bundle ("DATABASE/phase3"), and the certificate/report template set ("reports PDFs") — so that every functionality actually implemented in the codebase is captured as a verifiable requirement.

The intended audience includes SAC-ISRO project stakeholders, software quality assurance / certification reviewers, system administrators, maintenance engineers who will operate the deployed system, and any future development team tasked with extending or re-implementing the system.

1.2 Scope

CMCMIS digitizes the full lifecycle of calibration, maintenance, and repair of laboratory/test equipment and instruments at SAC-ISRO. In scope are:

- Raising, approving, and tracking Job Requests for calibration, repair, preventive maintenance, or installation of equipment.
- Conversion of approved Job Requests into Job Cards, and execution of Job Cards through Calibration, Maintenance, and Repair workflows with task checklists, observation/adjustment recording, spares consumption, and supporting-document attachment.
- Generation of statutory calibration certificates (NABL-accredited and Non-NABL formats), Job Closing Forms, and Job Request Forms (JRF) as PDF documents using both programmatic PDF generation (PDFKit) and LaTeX-authored master templates.
- Master-data management for Equipment, Employees, Vendors/Contractors, Departments, Sections, Projects, Task Library, and Checklist Library, including a formal change-request/approval workflow for correcting master data after initial verification.
- Role-based access control (RBAC) across five system roles with section/lane-level data scoping, JWT-based authentication, refresh-token rotation with theft detection, and an SSO login path against an employee directory.
- Operational scheduling (calendar) of lab activities with ICS calendar export, procurement tracking (purchase orders, spare parts inventory and re-ordering), management dashboards/KPIs, ad-hoc analytics, canned PDF/CSV reports, in-app notifications, a cross-entity quick-search ("Inquiry") facility, and a full, immutable audit trail of every state-changing action.

Out of scope of the delivered codebase (and therefore of this SRS, except where noted as an assumption/dependency): integration with instrument firmware/IoT telemetry, financial/accounting/ERP integration beyond purchase-order tracking, and a native mobile application (the system is delivered as a responsive web application only).

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
CMCMIS	Calibration & Maintenance Computerized Management Information System (the product covered by this SRS)
SAC	Space Applications Centre, ISRO
JR / JRF	Job Request / Job Request Form
JC	Job Card — the operational work-order generated from an approved Job Request
T&ME	Test & Measuring Equipment (a lab/section in the org structure)
F&PE	Fabrication & Process Equipment (a lab/section in the org structure)
NABL	National Accreditation Board for Testing and Calibration Laboratories
RBAC	Role-Based Access Control
JWT	JSON Web Token
SSO	Single Sign-On
KPI	Key Performance Indicator
ICS	iCalendar file format (RFC 5545) used for calendar export
PO	Purchase Order
RLS	(In this document) Row-Level (data) Scoping — restricting visible rows to a user's lane/section
Lane	An operational scoping unit (e.g. T&ME vs F&PE) used to restrict a user's visible data
MDC	Master Data Correction — the formal workflow for amending verified master records

1.4 References

- Source code baseline: SOFTWARE CODE/BE (Node.js/Express backend), SOFTWARE CODE/FE (React/Vite frontend), SOFTWARE CODE/DATABASE/phase3 (MySQL migration bundle), SOFTWARE CODE/reports PDFs (PDF/LaTeX certificate templates).
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications (structural template followed by this document).
- ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering.
- NABL accreditation documentation requirements (referenced by the NABL/Non-NABL certificate templates found in reports PDFs/latex).

1.5 Document Overview

Section 2 gives the overall product description, user classes, and operating environment. Section 3 documents the system and software architecture, technology stack, and database schema as implemented. Section 4 enumerates functional requirements module-by-module, each traceable to the corresponding backend module and frontend page in the codebase. Section 5 documents external interfaces (UI screens and the complete REST API). Section 6 documents data requirements and the relational schema. Section 7 documents non-functional requirements. Section 8 documents other/regulatory requirements. The appendices provide the complete API endpoint inventory, the database table inventory, the role-permission matrix, and the third-party library bill of materials.

2. Overall Description

2.1 Product Perspective

CMCMIS is a self-contained, three-tier web application:

- Presentation tier — a single-page application (SPA) built with React 18, served as static assets (FE/dist) behind an Nginx reverse proxy (SOFTWARE CODE/nginx.conf).
- Application tier — a Node.js/Express REST API (BE/src) implementing authentication, RBAC, business workflows, and PDF generation, structured as one module per business capability (controller → service → repository layering).
- Data tier — a MySQL relational database, evolved from a pre-existing legacy schema ("cmms_*" tables) via an idempotent, checksum-tracked migration bundle (DATABASE/phase3) that adds 40+ new tables, alters legacy tables in place, and seeds roles, permissions, lookups, and bootstrap admin accounts.

The product supersedes a manual / partially-legacy paper-and-spreadsheet process for raising calibration/maintenance job requests and recording job-card outcomes, while preserving and extending the pre-existing legacy MySQL master tables (e.g. equipment, employee, vendor masters) rather than replacing them outright.

2.2 Product Functions (Summary)

- Secure login (password or SSO) with JWT access/refresh tokens and full login audit trail.
- Role-based dashboards and KPI widgets tailored to each of the five user roles.
- Equipment master register: registration, search, verification, calibration-due tracking, condemnation, bulk calibration-done marking.
- Job Request lifecycle: raise → submit → approve/reject → convert to Job Card → cancel, with full status history and exportable PDF list/forms.
- Job Card lifecycle: start work → record calibration/maintenance/repair details → attach documents → mark complete → verify & close → (optionally) reopen, with type-specific sub-workflows for Calibration, Maintenance, and Repair job cards.
- Task checklist application to job cards from a managed Task Library / Checklist Library.
- Spares consumption tracking against job cards, linked to the spare-parts inventory.
- Generation of NABL and Non-NABL calibration certificates, Job Closing Forms, and JRF/Job Card detail PDFs from LaTeX-derived templates rendered through PDFKit.
- Master Data Correction workflow allowing non-admin users to request edits to verified master data, routed to Super Admin for approval/rejection.
- Scheduling/calendar of lab activities with single-event and full-calendar ICS export.
- Procurement: purchase order creation/tracking and spare-parts inventory management with re-order initiation.
- Canned operational reports (calibration-due, pending jobs, equipment utilization, engineer summary, job-card summary, job-request summary) in both JSON (for in-app display) and PDF export form.
- Ad-hoc analytics module with dynamically registered metrics and CSV export, plus a dedicated lab-capacity analytic.
- Cross-entity "Inquiry" quick search across vendors, products, job cards, and instruments.

- In-app notification center with unread-count badge and emitter/recipient-resolution architecture.
- Full, queryable, exportable audit log of every state-changing action with before/after field-level change capture.
- Administration console: user-account lifecycle (activate/deactivate/force-logout/role change), employee master CRUD with account provisioning, equipment-verification gate, job-request-terms management, dynamic projects/tasks/checklists CRUD, and master-data-correction review queue.

2.3 User Classes and Characteristics

The system implements exactly five system roles (locked, seeded by migration 005__seed_roles.sql), enforced through a 40-permission RBAC matrix (006/007) plus section/lane-level row scoping (110/700 series migrations):

Role Code	Description	Characteristic Capabilities
SUPER_ADMIN	Master data + RBAC management + system integrity oversight (top-tier admin)	Full read/write across all modules; manages users, employees, terms, projects, tasks, checklists, equipment verification, and master-data corrections; sole role permitted to verify newly registered equipment and bulk-verify operations.
LAB_IN_CHARGE	Approves job requests, assigns engineers, verifies completed work, manages schedules	Approves/rejects Job Requests, converts them to Job Cards, verifies & closes Job Cards, manages the lab schedule, scoped to their assigned lane/section (T&ME or F&PE).
LAB_ENGINEER	Executes assigned jobs, fills job cards, records observations, generates PDFs	Starts work, records calibration/maintenance/repair details, attaches documents, marks jobs complete, generates certificates — scoped to assigned jobs/lane.
NORMAL_USER	Raises job requests, registers equipment, tracks own requests	Creates Job Requests and equipment registrations, tracks only their own submissions, raises master-data correction requests.
VIEW_ONLY	Read all data; no write actions ever; auditor / management oversight	Read access across reports, dashboards, equipment, job requests/cards, and audit trail; cannot perform any create/update/delete action — enforced both client-side (lib/permissions.js) and server-side (authorize middleware + role_permissions grants).

All session-affecting role changes (role re-assignment, activation/deactivation, forced logout) are immediately enforced via a token_version JWT claim that the backend bumps whenever a Super Admin alters a user — so a live, already-issued access token loses its elevated/legacy permissions on the very next request rather than only after expiry.

2.4 Operating Environment

- Server runtime: Node.js ≥ 18 (per BE/package.json "engines").
- Database: MySQL (accessed via the mysql2 driver), retaining and extending a pre-existing legacy schema.
- Web server / reverse proxy: Nginx (SOFTWARE CODE/nginx.conf) serving the built FE static assets and proxying /api traffic to the Node backend.
- Client: any modern evergreen browser supporting ES2020+ and the React 18 / Vite build output; the UI is built with Tailwind CSS and is responsive.
- Development tooling: Vite 5 dev server for the frontend; nodemon for backend hot-reload in development.

2.5 Design and Implementation Constraints

- The backend must operate over the "sealed" Phase-3 database — i.e. it consumes the schema produced by the migration bundle and must not assume a green-field schema; several legacy "cmms_*" tables are preserved and only ALTERed, never dropped, to avoid breaking pre-existing data.
- Exactly five system roles are locked at the schema level (is_system=1) and are not intended to be end-user-extensible through the UI; permission grants are managed via the role_permissions grant matrix, not ad-hoc per-user flags.
- The HTTP middleware pipeline is a fixed, documented 13-step order (helmet → cors → compression → json body parsing → cookie-parser → pino-http logging → rate-limit → authenticate → authorize → row-level scope → schema validation → controller → notFound/errorHandler) and new middleware must be inserted at the marked points rather than reordering existing steps.
- All request bodies are capped at 1 MB (express.json({ limit: "1mb" })).
- Refresh tokens are never stored raw; only their SHA-256 hash is persisted, enabling replay/theft detection without retaining the bearer secret.
- PDF certificate layouts are authored first in LaTeX (reports PDFs/latex/*.tex) as the canonical visual reference and then re-implemented as PDFKit template modules (BE/src/modules/pdf/templates) for at-runtime generation — the two must be kept visually consistent.

2.6 Assumptions and Dependencies

- A reachable MySQL instance, already containing (or migrated to contain) the legacy "cmms_*" master tables that the Phase-3 bundle alters and back-fills.
- Network access from the application server to the database and, where SSO is used, to the employee SSO directory data source that seeds employee_sso_directory / employee_sso_heads.
- Environment configuration is supplied via .env / process environment and validated at boot through envalid (BE/src/config/env.js); the process exits if required variables are missing or malformed.

- Generated PDF certificates and forms assume the presence of the ISRO, NABL, and SAC logo image assets bundled under reports PDFs/logo and BE/src/assets.

3. System Architecture & Technology Stack

3.1 High-Level Architecture

CMCMIS follows a classic three-tier architecture with a clear module boundary inside the backend:

- Frontend (FE): React 18 SPA, built with Vite, styled with Tailwind CSS v3, communicating with the backend exclusively over a versioned JSON REST API via Axios, with TanStack Query managing server-state caching/invalidation and Zustand managing local/UI state (e.g., auth context).
- Backend (BE): Express 4 application. Each business capability is implemented as a self-contained module under BE/src/modules/<name>/ following a consistent four-layer pattern: <name>.routes.js (HTTP routing + middleware wiring) → <name>.controller.js (request/response shaping) → <name>.service.js (business rules, transactions, orchestration) → <name>.repo.js (parameterised SQL via mysql2). Complex domains additionally define <name>.stateMachine.js (e.g. Job Requests, Job Cards, Admin Users, Schedules) and <name>.validators.js (Zod schemas).
- Database (DATABASE): MySQL, structured as a legacy core (pre-existing "cmms_*" tables for equipment, employees, departments/sections, vendors, etc.) plus a Phase-3 migration bundle that introduces RBAC, audit, scheduling, procurement, notifications, master-data-correction, SSO, and job-card child-detail tables, all applied through an idempotent Node.js migration runner with checksum tracking (schema_migrations).

3.2 Backend Technology Stack

Package	Version	Purpose
express	^4.19.2	HTTP server and routing framework
mysql2	^3.11.0	MySQL driver (promise + pooled connections)
jsonwebtoken	^9.0.2	Signing/verifying JWT access & refresh tokens
bcryptjs	^2.4.3	Password hashing for local (non-SSO) accounts
zod	^3.23.0	Schema validation for request payloads (validate.js middleware)
helmet	^7.1.0	Security HTTP headers (CSP, HSTS, etc.)
cors	^2.8.5	Cross-Origin Resource Sharing policy enforcement
compression	^1.7.4	Gzip response compression
cookie-parser	^1.4.7	Parsing the refresh-token cookie
express-rate-limit	^7.4.0	Rate limiting on authentication endpoints

Package	Version	Purpose
pino / pino-http / pino-pretty	^9.4.0 / ^10.3.0 / ^11.2.0	Structured, per-request logging
multer	^1.4.5-lts.1	Multipart/form-data handling for document uploads
pdfkit	^0.18.0	Programmatic PDF generation for certificates, forms, and reports
dayjs	^1.11.13	Date/time manipulation and formatting
uuid	^9.0.1	Unique identifier generation
dotenv	^16.4.5	Loading environment variables from .env
envalid	^8.0.0	Validating/typing environment variables at boot
nodemon	^3.1.14	Development-time auto-restart (dev dependency)

3.3 Frontend Technology Stack

Package	Version	Purpose
react / react-dom	^18.3.1	Core UI library and DOM renderer
vite	^5.4.8	Dev server and production bundler
@vitejs/plugin-react	^4.3.2	Vite React plugin (Fast Refresh, JSX)
react-router-dom	^6.27.0	Client-side routing (route table in App.jsx)
@tanstack/react-query	^5.100.11	Server-state fetching/caching/invalidation
@tanstack/react-table	^8.21.3	Headless data-table logic for list/grid pages
axios	^1.7.7	HTTP client (api-client.js) with interceptors for auth/refresh
zustand	^5.0.13	Lightweight client-side state management
react-hook-form	^7.53.0	Form state management
@hookform/resolvers	^3.9.0	Bridges react-hook-form to Zod schemas
zod	^3.23.0	Client-side form/payload schema validation
dayjs	^1.11.20	Date/time manipulation and formatting

Package	Version	Purpose
recharts	^3.8.1	Charting library for dashboards/analytics
lucide-react	^0.454.0	Icon set
clsx	^2.1.1	Conditional CSS class composition
sonner	^2.0.7	Toast/notification UI feedback
tailwindcss	^3.4.13	Utility-first CSS framework
@tailwindcss/forms	^0.5.9	Form-control styling reset/plugin
postcss / autoprefixer	^8.4.47 / ^10.4.20	CSS post-processing pipeline

3.4 Backend Request Pipeline (Locked 13-Step Order)

Every incoming HTTP request to the Express app (BE/src/server.js) passes through the following fixed pipeline:

1. helmet() — security response headers
2. cors() — origin allow-listing, rejects foreign origins
3. compression() — gzip the response body
4. express.json({ limit: "1mb" }) — parse the JSON request body
5. cookie-parser() — parse the refresh-token cookie
6. pino-http() — emit one structured log line per request (with secret redaction)
7. rateLimit() — applied per-router, notably on /api/auth/* to throttle credential-guessing
8. authenticate() — verifies the JWT access token and attaches req.user
9. authorize(permission) — checks the caller's permission set against the route's required permission
10. rowLevelScope() — narrows queries to the caller's lane/section scope (Phase 5+ capability)
11. validate(schema) — Zod-schema request validation, applied per-route via a validator factory
12. controller — the route's business logic handler
13. notFoundHandler + errorHandler — always last; produce uniform 404 / error JSON responses

3.5 Module Inventory (Backend)

The backend implements the following 23 business modules under BE/src/modules, each independently routable and independently testable:

Module	Responsibility
auth	Authentication: password login, SSO employee login, password reset, JWT refresh rotation, logout
users	Lightweight "who am I" endpoint for the logged-in session
adminUsers	Super-Admin user-account lifecycle management

Module	Responsibility
	(role, activation, force-logout)
employees	Employee master CRUD + login-account provisioning
equipment	Equipment master register, verification, condemnation, calibration-due bulk actions
jobRequests	Job Request lifecycle (raise → submit → approve/reject → convert → cancel)
jobRequestTerms	Configurable terms & conditions text shown/attached to Job Requests
jobCards	Job Card lifecycle (core) plus five sub-modules: calibration, maintenance, repair, spares, documents, taskChecklist
tasks	Task-library CRUD (atomic checklist items)
checklists	Checklist-master CRUD and application of a checklist template to a job card
masterDataCorrections	Formal request/approve/reject workflow for amending verified master data
procurement	Purchase orders and spare-parts inventory, including re-order initiation
projects	Dynamic project master CRUD (used as a job-request/job-card dimension)
schedule	Lab activity scheduling, status state machine, and ICS calendar export
notifications	In-app notification feed, unread counters, emitter + recipient-resolution
reports	Six canned operational reports, each with a JSON and a PDF variant
analytics	Dynamically registered analytics metrics with JSON + CSV export, plus lab-capacity analytic
dashboard	Role-scoped KPI summary for the landing dashboard
inquiry	Cross-entity quick search (vendors, products, job cards, instruments)
audit	Full audit-log query, filter-facet discovery, and export
lookups	Shared reference-data lookups (divisions, projects, engineers, task library, etc.)
pdf	Standalone certificate/details PDF endpoints mounted ahead of the main job-request/job-card routers
masterDataCorrections / users / employees (admin) etc.	(see above) — admin-scoped variants are mounted under /api/admin/*

3.6 Deployment Architecture

Production deployment serves the Vite-built static FE bundle and reverse-proxies API traffic through Nginx (SOFTWARE CODE/nginx.conf) to the Node.js Express process, which in turn connects to the MySQL database. The backend honours the X-Forwarded-For header (Express "trust proxy") so that client IP addresses are correctly attributed in rate-limiting and audit/login-audit logging when running behind Nginx.

4. Functional Requirements

Each requirement is identified as FR-<Module>-<n>. Requirements are derived directly from the implemented controller/service logic, state machines, and validators in the codebase, and from the corresponding FE pages under FE/src/pages.

4.1 Authentication & Session Management

Corresponding frontend page(s): pages/home (login), lib/auth-context.jsx

FR-AUTH-1: The system shall authenticate users via employee ID + password against the users table, with the password hash verified using bcrypt (auth.service.js: login()).

FR-AUTH-2: The system shall support an SSO login path (loginSsoByEmployeeId) that authenticates against the employee_sso_directory / employee_sso_heads tables without a local password, recording the chosen auth source ("PASSWORD" vs "SSO") on the issued refresh token.

FR-AUTH-3: On successful login, the system shall issue a short-lived JWT access token (containing employee id, role code, permission list, and lane scopes) and a longer-lived JWT refresh token, and persist a SHA-256 hash (never the raw token) of the refresh token in refresh_tokens (or sso_refresh_tokens for SSO sessions) with an expiry timestamp.

FR-AUTH-4: The system shall record every login attempt (success and failure) in login_audit with timestamp, IP address, and user agent.

FR-AUTH-5: The system shall expose a /auth/refresh endpoint that rotates the refresh token on every use: it issues a new access+refresh pair, invalidates the old refresh token, and re-embeds the caller's current permission set so that a role change is reflected without forcing a fresh login.

FR-AUTH-6: The system shall detect refresh-token replay/theft: if a refresh JWT has a valid signature but its hash is not found among the user's persisted refresh tokens (i.e., it was already rotated away), the system shall treat this as a compromise indicator and revoke every refresh token belonging to that user, forcing re-authentication for both the attacker and the legitimate user.

FR-AUTH-7: The system shall support a forgot-password flow (/auth/forgot-password).

FR-AUTH-8: The system shall support an explicit logout (/auth/logout) that revokes the presented refresh token and is idempotent (repeated calls do not error).

FR-AUTH-9: The system shall embed a token_version claim in access tokens; any Super-Admin action that changes a user's role, status, or forces logout shall increment the stored token_version, causing all previously issued access tokens for that user to be rejected by the authenticate middleware on their next use.

FR-AUTH-10: The system shall rate-limit the /auth/* endpoints to mitigate credential-stuffing/brute-force attempts.

4.2 User & Role Management (RBAC)

Corresponding frontend page(s): pages/admin (Users tab)

FR-USER-1: The system shall provide exactly five system roles (SUPER_ADMIN, LAB_IN_CHARGE, LAB_ENGINEER, NORMAL_USER, VIEW_ONLY), seeded as immutable system rows (is_system = 1).

FR-USER-2: The system shall enforce a 40-entry atomic permission catalogue, granted to roles via a role_permissions matrix (SUPER_ADMIN is granted every permission; other roles receive a curated subset).

FR-USER-3: The system shall allow a Super Admin to list all user accounts, view a single account's detail and its role/status change history, change a user's role, activate or deactivate an account, and force-logout a user (revoking all of that user's active refresh tokens / bumping token_version).

FR-USER-4: The system shall scope certain users to an operational "lane" (e.g., T&ME or F&PE) via user_lane_scopes, restricting the rows that user can read or act on in lane-aware modules (job requests, job cards, equipment, schedules).

FR-USER-5: The system shall record every role/permission-affecting change in user_role_history for audit purposes.

4.3 Employee Master Management

Corresponding frontend page(s): pages/admin (Employees tab)

FR-EMPL-1: The system shall allow Super Admins to create, list, view, update, and delete employee master records.

FR-EMPL-2: The system shall allow provisioning a system login account for an existing employee record via a dedicated "create account" action, which establishes the initial credentials and default role assignment.

FR-EMPL-3: The system shall maintain departments and sections as first-class master tables (seeded with the TIMCD department and the T&ME / F&PE sections) to which employees and job-request submitters are associated.

4.4 Equipment Master Management

Corresponding frontend page(s): pages/equipment (list, detail, new)

FR-EQUI-1: The system shall allow registration of new equipment with type, make, division, project, and accessory metadata (validated against the types/makes/divisions/projects/accessories lookup endpoints).

FR-EQUI-2: The system shall allow searching and listing equipment with pagination and filters, and exporting the filtered list as a PDF register.

FR-EQUI-3: The system shall track each equipment item's calibration-due status and allow Super Admins to bulk-mark a selected set of equipment as calibration-done in a single action (bulk-cal-done).

FR-EQUI-4: The system shall restrict equipment-record verification to the Super Admin role only (per the dedicated 750__equipment_verification_super_admin_only migration), recording the verifying user and timestamp.

FR-EQUI-5: The system shall allow an equipment item to be condemned/decommissioned, transitioning its lifecycle status and recording the action.

FR-EQUI-6: The system shall maintain a full equipment_status_history audit trail of every status transition an equipment record undergoes.

FR-EQUI-7: The system shall allow soft/hard deletion of equipment records subject to permission checks.

4.5 Job Request Management

Corresponding frontend page(s): pages/jobRequests (list, detail, new)

FR-JOB-1: The system shall allow a Normal User (or higher role) to raise a Job Request specifying equipment, requested work type (calibration / repair / maintenance / installation), accessories, project/division context, and free-text remarks.

FR-JOB-2: The system shall allow a Job Request to be saved as a draft and explicitly submitted for approval (/:id/submit), transitioning it through a documented status state machine (jobRequests.stateMachine.js).

FR-JOB-3: The system shall allow a Lab In-Charge to approve and convert a Job Request into a Job Card (/:id/convert), or to reject it with a reason (/:id/reject).

FR-JOB-4: The system shall allow bulk verification of multiple Job Requests in a single action (bulk-verify-all) for Lab In-Charge efficiency.

FR-JOB-5: The system shall allow the requester (or an authorised role) to cancel a Job Request prior to conversion (/:id/cancel), and to permanently delete a Job Request where permitted.

FR-JOB-6: The system shall maintain a job_request_status_history table capturing every status transition with actor, timestamp, and optional remark, exposed via the /:id/history endpoint.

FR-JOB-7: The system shall record job-request-level accessory associations (job_request_accessories) and a configurable set of standard terms & conditions (job_request_terms) presentable on the JRF.

FR-JOB-8: The system shall allow exporting the Job Request list as a PDF and generating an individual JRF "details.pdf" document for a specific request.

FR-JOB-9: The system shall apply row-level lane scoping so that Lab In-Charge/Engineer users only see Job Requests belonging to their assigned section/lane, while Normal Users only see their own submissions.

4.6 Job Card Management (Core + Calibration/Maintenance/Repair)

Corresponding frontend page(s): pages/jobCards (list, detail)

FR-JOB-1: The system shall create a Job Card automatically when a Job Request is converted, carrying forward equipment, requester, and work-type context.

FR-JOB-2: The system shall drive each Job Card through a documented status state machine (jobCards.stateMachine.js): assigned → work started → completed → verified/closed, with an explicit reopen transition available to authorised roles.

FR-JOB-3: The system shall allow a Lab Engineer to start work on an assigned Job Card (/:id/start-work) and to mark it complete (/:id/mark-complete) once all required sub-records are filled in.

FR-JOB-4: The system shall allow a Lab In-Charge to verify and close a completed Job Card (/:id/verify-close), and to reopen a previously closed Job Card (/:id/reopen) if rework is required.

FR-JOB-5: The system shall branch the data captured on a Job Card by work-type into three specialised sub-workflows, each with its own dedicated repository tables and endpoints:

- Calibration sub-workflow — records equipment used for calibration (jc_calibration_equipment_used) and calibration adjustment/observation rows (jc_calibration_adjustments, jc_observations_readings) via dedicated equipment-rows and adjustment-rows endpoints.

    - Maintenance sub-workflow — records preventive/corrective maintenance detail rows (jc_maintenance_details).
    - Repair sub-workflow — records equipment used during repair (jc_repair_equipment_used) via dedicated equipment-rows endpoints.

1. The system shall allow a checklist template (from the Checklist Library) to be applied to a Job Card, generating per-task checklist rows (jc_task_checklist / jc_calibration_task_checklist) that the engineer marks complete with pass/fail/observation values.
2. The system shall allow spares consumed during a job to be recorded against the Job Card (jc_spares_used), linked to the spare-parts inventory.
3. The system shall allow supporting documents (e.g., scanned forms, photographs) to be uploaded to and downloaded from a Job Card (jc_documents) via multipart file upload (multer), and deleted by an authorised user.
4. The system shall maintain a job_card_status_history table of every status transition, exposed via the /:id/history endpoint.
5. The system shall generate, per Job Card: a generic Job Card PDF, a Job Card "details.pdf", a calibration certificate PDF, a generic certificate PDF, an NABL-accredited certificate PDF, and a Non-NABL certificate PDF — each rendered through the BE/src/modules/pdf templates and mirroring the canonical LaTeX masters in reports PDFs/latex.
6. The system shall allow exporting the Job Card list as a PDF, filtered by the caller's lane scope.

4.7 Task Library, Checklist Library & Projects (Configuration Masters)

Corresponding frontend page(s): pages/admin (Tasks, Checklists, Projects tabs)

FR-TASK-1: The system shall allow Super Admins to maintain a Task Library (task_library) of reusable atomic checklist items (create/update/delete).

FR-TASK-2: The system shall allow Super Admins to maintain Checklist Masters (checklists_master / checklist_master_tasks) — named, ordered groupings of Task Library items applicable to specific equipment types — and to apply a checklist master directly onto a given Job Card.

FR-TASK-3: The system shall allow Super Admins to maintain a dynamic Projects master (used as a selectable dimension on Job Requests/Job Cards) with full CRUD.

4.8 Spares & Documents

Corresponding frontend page(s): pages/jobCards (detail tabs)

FR-SPAR-1: The system shall allow listing, adding, updating, and removing spare-parts-used rows against a specific Job Card.

FR-SPAR-2: The system shall allow listing, uploading, retrieving, and deleting documents attached to a specific Job Card, with uploaded files persisted under BE/storage/job-cards.

4.9 Procurement Management

Corresponding frontend page(s): pages/procurement

FR-PROC-1: The system shall allow creating, listing, retrieving, and updating Purchase Orders, each composed of one or more purchase_order_items.

FR-PROC-2: The system shall allow exporting the Purchase Order list.

FR-PROC-3: The system shall maintain a Spare Parts inventory master (spare_parts) supporting create, list, retrieve, update, and export operations.

FR-PROC-4: The system shall allow initiating a re-order for a given spare part (/spare-parts/:id/order), e.g. when stock falls below a threshold.

4.10 Schedule Management

Corresponding frontend page(s): pages/schedule

FR-SCHE-1: The system shall allow creating, listing, retrieving, updating, and deleting scheduled lab activities (schedules table) presented as a calendar.

FR-SCHE-2: The system shall drive each scheduled activity through a status state machine (schedule.stateMachine.js) via the /:id/status endpoint, with every transition recorded in schedule_status_history.

FR-SCHE-3: The system shall export the full calendar as a single .ics file (export.ics) and export an individual scheduled item as its own .ics file (/:id/ics) for import into external calendar clients (schedule.ics.js).

4.11 Master Data Correction Workflow

Corresponding frontend page(s): pages/masterDataCorrections

FR-MAST-1: The system shall allow a non-admin user to request a correction to a verified master record (e.g., an equipment field that can no longer be edited directly once verified) by submitting a master_data_correction_requests row referencing the target entity, field(s), and proposed new value(s).

FR-MAST-2: The system shall expose a /context endpoint returning the editable fields and current values for the entity being corrected, so the FE can render an accurate before/after diff.

FR-MAST-3: The system shall route every correction request to the Super Admin review queue, who may approve it (applying the change to the underlying master record) or reject it (with the request left unmodified), both actions being permission-gated (806__master_data_correction_permissions.sql) and fully audited.

4.12 Notifications

Corresponding frontend page(s): pages/notifications, components/notifications

FR-NOTI-1: The system shall generate in-app notifications for events such as job-request approval/rejection, job-card assignment, and master-data-correction decisions, using an emitter/recipient-resolution pattern (notifications.emitter.js, notifications.recipients.js) that determines the correct recipient set per event type (e.g., the requester, the assigned engineer, the lane's Lab In-Charge).

FR-NOTI-2: The system shall expose an unread-count endpoint for badge display and allow marking a single notification or all notifications as read.

4.13 Reports & PDF Generation

Corresponding frontend page(s): pages/reports (ReportsLanding, ReportFilters, ReportTable, ExportPanel, charts)

FR-REPO-1: The system shall provide six canned operational reports — Calibration-Due, Pending Jobs, Equipment Utilization, Engineer Summary, Job-Card Summary, and Job-Request Summary — each available as a filterable JSON dataset for in-app display (tables, summary tiles, and charts) and as a PDF export.

FR-REPO-2: The system shall scope report data to the caller's row-level lane (resolveRowScope) and append a metadata block (report id/title, generated-by, generated-at, applied filters) to every report response for traceability (buildMeta).

FR-REPO-3: The system shall render report PDFs using the shared ISRO header template (_isroHeader.js) for consistent institutional branding.

4.14 Analytics & Dashboard

Corresponding frontend page(s): pages/analytics, pages/dashboard

FR-ANAL-1: The system shall provide a role-scoped KPI summary (/dashboard/kpis) for the landing dashboard, surfaced through StandardKpiCard components and recharts visualisations.

FR-ANAL-2: The system shall provide a dynamically registered set of analytics metrics, each automatically exposing both a JSON endpoint and a corresponding /<metric>/csv export endpoint (analytics.routes.js metric-registration loop), enabling new analytics to be added without bespoke routing code.

FR-ANAL-3: The system shall provide a dedicated Lab Capacity analytic (/analytics/lab-capacity) correlating engineer/lab throughput against incoming job volume.

4.15 Inquiry Module

Corresponding frontend page(s): pages/inquiry

FR-INQU-1: The system shall provide a cross-entity quick-search facility allowing a user to search Vendors (cmms_cont_mst), Products/spare catalogue items, Job Cards, and Instruments/equipment from a single search interface.

4.16 Audit Trail

Corresponding frontend page(s): pages/audit

FR-AUDI-1: The system shall record every state-changing action across all modules into audit_log, with field-level before/after values captured in audit_log_changes.

FR-AUDI-2: The system shall provide a filterable, paginated audit-log browser, a /filters endpoint to discover available filter facets (actor, module, action type, date range), and an /export endpoint that itself logs the export event to export_audit for non-repudiation.

FR-AUDI-3: The system shall allow drilling into a single audit entry to see its complete field-level change set.

4.17 Lookups & Shared Reference Data

Corresponding frontend page(s): used across most pages as dropdown sources

FR-LOOK-1: The system shall expose shared, cacheable lookup endpoints for divisions, projects, submitter context (defaulting a Job Request submitter's own section/lane), equipment type-ahead search, equipment accessories, eligible lab engineers, task-library entries, and calibration personnel — consumed throughout the FE to populate dropdowns and auto-complete fields consistently.

5. External Interface Requirements

5.1 User Interfaces

The frontend (React Router route table, FE/src/App.jsx) exposes the following screens, each backed by a corresponding page module under FE/src/pages:

Route	Screen / Purpose
/login	Login screen (password and SSO)
/home, /dashboard	Landing page and role-scoped KPI dashboard
/profile	Current user profile
/about, /user-guide, /view-only-guide, /lab-incharge-guide, /lab-engineer-guide, /super-admin-guide	Static help/about and role-specific user guides
/equipment, /equipment/new, /equipment/:id	Equipment register list, registration form, and detail view
/job-requests, /job-requests/new, /job-requests/:id	Job Request list, creation form, and detail/approval view
/master-data-corrections/new	Master-data correction request form
/conversion	Job-Request → Job-Card conversion workflow screen
/job-cards, /job-cards/:id	Job Card list and detail (with calibration/maintenance/repair/checklist/spares/documents tabs)
/schedule	Calendar view of scheduled lab activities
/procurement	Purchase orders and spare-parts inventory management
/inquiry	Cross-entity quick search
/reports	Canned report browser with filters, tables, summary tiles, charts, and export panel
/analytics	Ad-hoc analytics dashboards
/notifications	Notification center
/admin/users	User-account administration
/admin/employees, /admin/employees/new, /admin/employees/:id/edit	Employee master administration
/admin/equipment-verification	Equipment-verification queue (Super Admin only)
/admin/terms	Job-request terms & conditions administration
/admin/projects, /admin/tasks, /admin/checklists	Projects / Task Library / Checklist Library administration
/admin/master-data-corrections	Master-data-correction review queue
/admin/lab-capacity	Lab capacity configuration/analytic
/audit	Audit-log browser

Shared UI building blocks (FE/src/components) include a persistent Sidebar/TopBar/Layout shell, a reusable DataTable with Pagination built on TanStack Table, a ProtectedRoute guard that enforces the route's required permission against the authenticated user's permission set before rendering, StatusPill badges for workflow states, and a notifications widget.

5.2 API Interfaces

All endpoints are mounted under the configurable API base path (env.API_BASE_PATH, conventionally /api) and, except for /api/auth/* and the public health/PDF download routes, require a valid bearer JWT access token plus the specific permission grant checked by the authorize middleware. The complete endpoint inventory, grouped by module, is provided in Appendix A.

Request/response bodies are JSON (application/json) except for: (a) file upload endpoints (multipart/form-data via multer), and (b) the */pdf, */export-pdf, *.pdf, and *.ics endpoints, which return binary PDF or text/calendar payloads with an appropriate Content-Disposition header for download.

5.3 Hardware Interfaces

CMCMIS has no direct hardware interface requirement; it does not connect to laboratory instruments, sensors, or controllers. All calibration/maintenance/repair data is entered by lab engineers through the web UI rather than ingested automatically from equipment.

5.4 Communication Interfaces

- HTTPS/HTTP over standard TCP/IP between the browser client and the Nginx-fronted application server.
- Internal TCP connection from the Node.js backend to the MySQL database via the mysql2 connection pool.
- Outbound generated artefacts (PDFs, ICS files, CSV exports) are delivered over the same HTTPS connection as standard file downloads; no email/SMTP integration is present in the codebase.

6. Data Requirements

The relational schema is implemented in MySQL and evolved through the idempotent, checksum-tracked migration bundle in DATABASE/phase3/migrations (28 migration files plus 2 JS seeders, applied via a custom runner that records each applied file's SHA-256 in schema_migrations). The schema comprises the pre-existing legacy "cmms_*" master tables (equipment, employee, vendor/contractor, parameter/lookup masters — altered in place rather than replaced) plus 40+ new tables introduced by the bundle. The new tables are grouped below by functional area; the full table-by-table inventory is in Appendix B.

Functional Area	Tables
Identity & RBAC	departments, sections, roles, permissions, role_permissions, users, user_roles, user_role_history, user_lane_scopes, refresh_tokens, login_audit
SSO	employee_sso_directory, employee_sso_heads, sso_user_roles, sso_refresh_tokens
Job Request	job_request_status_history, job_request_accessories, job_request_terms
Job Card — Calibration	jc_calibration_equipment_used, jc_calibration_adjustments, jc_calibration_task_checklist, jc_observations_readings
Job Card — Maintenance / Repair	jc_maintenance_details, jc_repair_equipment_used
Job Card — shared	job_card_status_history, jc_task_checklist, jc_spares_used, jc_documents
Checklist / Task masters	task_library, checklists_master, checklist_master_tasks
Equipment	equipment_status_history
Vendor / Contractor	cmms_cont_mst (back-filled from legacy distinct manufacturer values)
Procurement	purchase_orders, purchase_order_items, spare_parts
Scheduling	schedules, schedule_status_history
Notifications	notifications
Master Data Correction	master_data_correction_requests
Operational Scoping	operational_lanes
Audit	audit_log, audit_log_changes, export_audit
Migration bookkeeping	schema_migrations

6.1 Key Entity Relationships

- A Job Request (1) converts to at most one Job Card (1), recorded via the conversion action and reflected in job-request status; a Job Card always references back to its originating Job Request and Equipment.
- A Job Card has exactly one active "type" sub-record set depending on work type: Calibration rows, Maintenance rows, or Repair rows — never more than one type concurrently for a given Job Card.
- A Job Card may have zero-to-many task checklist rows (optionally seeded from a Checklist Master), zero-to-many spares-used rows, and zero-to-many attached documents.
- Every Users row is associated with exactly one active Role (via user_roles) and may additionally carry zero-to-many lane scopes (user_lane_scopes) restricting visible data.
- Every state-changing write across every module produces a corresponding audit_log row, optionally paired with one-or-more audit_log_changes rows capturing the specific field-level diff.

6.2 Data Retention & Idempotency

Status-history tables (equipment_status_history, job_request_status_history, job_card_status_history, schedule_status_history, user_role_history) are append-only and never overwritten, preserving a complete chronological audit of every workflow transition. All migrations are written to be safely re-runnable (idempotent), using CREATE TABLE IF NOT EXISTS, INSERT IGNORE on primary/unique keys, an information_schema-driven safe-ALTER helper procedure, and a checksum-gated migration runner that skips files already applied.

7. Non-Functional Requirements

7.1 Security

- All passwords shall be stored as bcrypt hashes; raw passwords shall never be persisted or logged.
- All API access (other than login/refresh/forgot-password) shall require a valid, non-expired JWT access token, verified by the authenticate middleware on every request.
- Every protected route shall enforce a specific permission grant via the authorize middleware, sourced from the role_permissions matrix, in addition to row-level lane scoping where applicable.
- Refresh tokens shall be persisted only as a SHA-256 hash; presentation of a refresh token whose hash is not found among the user's valid, unexpired tokens shall trigger full session revocation for that user (anti-replay).
- HTTP responses shall carry hardened security headers via helmet() (e.g., HSTS, X-Content-Type-Options, CSP-related headers).
- Cross-origin requests shall be restricted by an explicit CORS allow-list; unrecognised origins shall be rejected.
- Authentication endpoints shall be rate-limited to mitigate brute-force/credential-stuffing attacks.
- Equipment verification and bulk-verification actions shall be restricted to the Super Admin role only, enforced at the permission-seed level (migration 750).
- Every state-changing action shall be attributable to an authenticated user and timestamp via the audit_log subsystem, and exports of sensitive data (audit log, reports) shall themselves be logged (export_audit).

7.2 Performance

- List/search endpoints (equipment, job requests, job cards, purchase orders, spare parts, audit log) shall support pagination to bound response payload size.
- Dedicated full-text and composite indexes shall be applied to high-traffic search columns (migrations 121__phase8_fulltext_indexes.sql, 790__optimize_search_indexes.sql) to keep list/search queries performant as data volume grows.
- HTTP responses shall be gzip-compressed (compression middleware) to reduce payload transfer time.
- Request bodies shall be capped at 1 MB to bound parsing cost and guard against oversized payload abuse.

7.3 Reliability & Availability

- Database schema changes shall be applied through an idempotent migration runner so that re-running the migration bundle (e.g., after a partial failure) never corrupts or duplicates data.
- The connectivity check performed at process boot (config/db.js) shall fail fast (exit the process) if the database is unreachable or misconfigured, rather than serving requests against a broken connection.

- The refresh-token rotation and theft-detection mechanism shall ensure that a compromised, already-rotated refresh token cannot be used to silently maintain unauthorized access.

7.4 Usability

- The UI shall present role-appropriate navigation and dashboards, hiding actions a user's role/permission set does not allow (ProtectedRoute + lib/permissions.js).
- The system shall provide role-specific in-app user guides (View-Only, Lab In-Charge, Lab Engineer, Super Admin) accessible from the application shell.
- Form validation shall be performed client-side (react-hook-form + Zod resolvers) ahead of submission, mirroring the server-side Zod validation, to give immediate, actionable feedback before a round-trip.
- Toast notifications (sonner) shall provide non-blocking success/error feedback for asynchronous actions.

7.5 Maintainability

- The backend shall maintain a one-module-per-capability structure with a consistent routes → controller → service → repo layering, so that business logic, HTTP concerns, and data access remain independently testable and replaceable.
- Complex, multi-state domains (Job Requests, Job Cards, Admin Users, Schedules) shall encapsulate their transition rules in a dedicated stateMachine module rather than scattering conditional logic across controllers.
- Database migrations shall be numbered, single-purpose, and accompanied by inline documentation of intent (as observed throughout DATABASE/phase3/migrations), enabling safe incremental evolution of the schema.

7.6 Scalability

- The mysql2 connection pool and stateless JWT-based authentication allow the Express application tier to be horizontally scaled behind a load balancer without server-side session affinity.
- The analytics module's dynamic metric-registration pattern allows new analytics endpoints to be added without structural changes to the routing layer.

7.7 Auditability & Compliance

- The system shall maintain a complete, field-level audit trail (audit_log / audit_log_changes) of all state-changing operations, queryable and exportable for internal/external compliance review.
- Generated calibration certificates shall be available in both NABL-accredited and Non-NABL formats, matching the corresponding LaTeX master templates, to support the laboratory's accreditation reporting obligations.

8. Other Requirements

8.1 Regulatory / Standards

Certificate generation must conform to the documentation conventions implied by NABL accreditation (separate NABL vs Non-NABL certificate templates are explicitly maintained, see reports PDFs/NABL_Certificate.pdf and NON-NABL_Certificate.pdf and their LaTeX masters). The system carries ISRO and SAC branding/logo assets on all generated official documents (reports PDfs/logo/isro-logo.png, sac-logo.png, nabl-logo.png).

8.2 Legacy Data Preservation

The Phase-3 migration bundle is explicitly designed to preserve all pre-existing legacy "cmms_*" master data — altering rather than dropping legacy tables, isolating unused legacy tables under a "_legacy_" prefix rather than deleting them (099__isolate_legacy_unused.sql) — so historical equipment, employee, and vendor records predating CMCMIS remain intact and queryable.

Appendix A: Complete REST API Endpoint Inventory

Derived directly from BE/src/modules/**/*.routes.js and the router mount points in BE/src/server.js. All paths are relative to the configured API base path (conventionally /api).

Auth — base: /api/auth

Method	Path	Description
POST	/login	Username/password login -> JWT access+refresh pair
POST	/sso/employee-login	SSO login via employee directory
POST	/forgot-password	Password reset request
POST	/refresh	Rotate refresh token, issue new access token
POST	/logout	Revoke refresh token

Users — base: /api

Method	Path	Description
GET	/me	Get current authenticated user profile + permissions

Equipment — base: /api/equipment

Method	Path	Description
GET	/types	List equipment type lookups
GET	/makes	List equipment make/manufacturer lookups
GET	/divisions	List division lookups
GET	/projects	List project lookups
GET	/export-pdf	Export equipment register as PDF
GET	/	List/search equipment (paginated, filterable)
POST	/	Register new equipment
POST	/bulk-cal-done	Bulk mark calibration-done for selected equipment
GET	/:id	Get equipment detail
PATCH	/:id	Update equipment
POST	/:id/verify	Verify equipment record (Super Admin only)
POST	/:id/condemn	Mark equipment condemned/decommissioned

Method	Path	Description
DELETE	/:id	Delete equipment

Job Requests — base: /api/job-requests

Method	Path	Description
GET	/	List job requests (filterable, role-scoped)
POST	/	Raise a new job request
GET	/export-pdf	Export job request list as PDF
POST	/:id/submit	Submit a draft request for approval
POST	/bulk-verify-all	Bulk verify multiple job requests
GET	/:id	Get job request detail
GET	/:id/history	Get status-change history
POST	/:id/convert	Convert approved request into a Job Card
POST	/:id/reject	Reject a job request
PATCH	/:id	Update job request
POST	/:id/cancel	Cancel a job request
DELETE	/:id	Delete a job request
GET	/:id/details.pdf	PDF: job request details (JRF form)

Master Data Corrections — base: /api/master-data-corrections

Method	Path	Description
GET	/context	Get correction-request context (fields, current values)
POST	/	Submit a master-data correction request
GET	/	List correction requests
POST	/:id/approve	Approve correction request (writes change to master record)
POST	/:id/reject	Reject correction request

Job Cards (core) — base: /api/job-cards

Method	Path	Description
GET	/	List job cards (role/lane scoped)
GET	/export-pdf	Export job card list as PDF
GET	/:id	Get job card detail

Method	Path	Description
GET	/:id/history	Get job card status history
PATCH	/:id	Update job card header fields
POST	/:id/start-work	Engineer starts work (state transition)
POST	/:id/mark-complete	Engineer marks job complete
POST	/:id/verify-close	Lab In-Charge verifies & closes job card
POST	/:id/reopen	Reopen a closed job card
GET	/:id/pdf	Generate generic job card PDF
GET	/:id/certificate.pdf	Generate calibration certificate PDF
GET	/:id/details.pdf	Generate job card details PDF
GET	/:id/certificates/nabl.pdf	Generate NABL-accredited certificate
GET	/:id/certificates/non-nabl.pdf	Generate Non-NABL certificate
GET	/:id/certificates/certificate.pdf	Generate generic certificate

Job Cards - Task Checklist — base: /api/job-cards/:id/tasks

Method	Path	Description
GET	/	List checklist tasks attached to a job card
POST	/	Add a checklist task
PATCH	/:taskId	Update task completion / result
DELETE	/:taskId	Remove a checklist task

Job Cards - Documents — base: /api/job-cards/:id/documents

Method	Path	Description
GET	/	List documents attached to a job card
POST	/	Upload a document (multer)
GET	/:docId	Download / view a document
DELETE	/:docId	Delete a document

Job Cards - Maintenance — base: /api/job-cards/:id/maintenance-rows

Method	Path	Description
GET	/	List maintenance detail rows
POST	/	Add a maintenance row
PATCH	/:rowId	Update a maintenance row

Method	Path	Description
DELETE	/:rowId	Delete a maintenance row

Job Cards - Spares — base: /api/job-cards/:id/spares-rows

Method	Path	Description
GET	/	List spares-used rows
POST	/	Add a spares-used row
PATCH	/:rowId	Update a spares-used row
DELETE	/:rowId	Delete a spares-used row

Job Cards - Calibration — base: /api/job-cards/:id/calibration

Method	Path	Description
GET	/equipment-rows	List calibration equipment-used rows
POST	/equipment-rows	Add equipment-used row
PATCH	/equipment-rows/:rowId	Update equipment-used row
DELETE	/equipment-rows/:rowId	Delete equipment-used row
GET	/adjustment-rows	List calibration adjustment/observation rows
POST	/adjustment-rows	Add adjustment row
PATCH	/adjustment-rows/:rowId	Update adjustment row
DELETE	/adjustment-rows/:rowId	Delete adjustment row

Job Cards - Repair — base: /api/job-cards/:id/repair

Method	Path	Description
GET	/equipment-rows	List repair equipment-used rows
POST	/equipment-rows	Add equipment-used row
PATCH	/equipment-rows/:rowId	Update equipment-used row
DELETE	/equipment-rows/:rowId	Delete equipment-used row

Lookups — base: /api/lookups

Method	Path	Description
GET	/divisions	Division lookup list
GET	/projects	Project lookup list
GET	/submitter-context	Context for the logged-in submitter (section/lane defaults)
GET	/equipment/search	Type-ahead equipment search
GET	/equipment/accessories	Accessories lookup for an equipment item

Method	Path	Description
GET	/engineers	Lab engineer lookup (for assignment dropdowns)
GET	/task-library	Task library lookup
GET	/calibration-people	Calibration personnel lookup

Job Request Terms — base: /api/job-request-terms

Method	Path	Description
GET	/	List active job-request terms & conditions
GET	/all	List all terms including inactive
POST	/	Create a new term
PUT	/:id	Update a term
DELETE	/:id	Delete/deactivate a term

Admin - Users — base: /api/admin/users

Method	Path	Description
GET	/	List system user accounts
GET	/:id	Get user account detail
GET	/:id/history	Get role/status change history
PATCH	/:id/role	Change a user's assigned role
PATCH	/:id/activate	Activate a user account
PATCH	/:id/deactivate	Deactivate a user account
POST	/:id/force-logout	Revoke all refresh tokens / bump token_version

Admin - Employees — base: /api/admin/employees

Method	Path	Description
GET	/	List employee master records
GET	/:id	Get employee detail
POST	/	Create employee record
PATCH	/:id	Update employee record
DELETE	/:id	Delete employee record
POST	/:id/create-account	Provision a login account for an employee

Dashboard — base: /api/dashboard

Method	Path	Description
GET	/kpis	Get role-scoped KPI summary

Method	Path	Description
		for the landing dashboard

Inquiry — base: /api/inquiry

Method	Path	Description
GET	/vendors	Search vendor master (cmms_cont_mst)
GET	/products	Search product/spare catalogue
GET	/job-cards	Quick search across job cards
GET	/instruments	Quick search across equipment/instruments

Notifications — base: /api/notifications

Method	Path	Description
GET	/	List notifications for current user
GET	/unread-count	Get unread notification count (for badge)
PATCH	/read-all	Mark all notifications read
PATCH	/:id/read	Mark one notification read

Reports — base: /api/reports

Method	Path	Description
GET	/calibration-due	Calibration-due report (JSON)
GET	/calibration-due/pdf	Calibration-due report (PDF)
GET	/pending-jobs	Pending-jobs report (JSON)
GET	/pending-jobs/pdf	Pending-jobs report (PDF)
GET	/equipment-utilization	Equipment utilization report (JSON)
GET	/equipment-utilization/pdf	Equipment utilization report (PDF)
GET	/engineer-summary	Engineer workload summary (JSON)
GET	/engineer-summary/pdf	Engineer workload summary (PDF)
GET	/job-card-summary	Job card summary report (JSON)
GET	/job-card-summary/pdf	Job card summary report (PDF)
GET	/job-request-summary	Job request summary report (JSON)
GET	/job-request-summary/pdf	Job request summary report

Method	Path	Description
		(PDF)

Analytics — base: /api/analytics

Method	Path	Description
GET	/<metric>	Dynamic analytics endpoints registered per metric (trend/aggregation JSON)
GET	/<metric>/csv	CSV export counterpart of each analytics metric
GET	/lab-capacity	Lab capacity analytics (utilization vs. throughput)

Schedules — base: /api/schedules

Method	Path	Description
GET	/	List scheduled activities (calendar feed)
GET	/export.ics	Export entire calendar as an .ics file
POST	/	Create a scheduled activity
GET	/:id	Get schedule item detail
PATCH	/:id	Update schedule item
POST	/:id/status	Transition schedule item status (state machine)
DELETE	/:id	Delete schedule item
GET	/:id/ics	Export single schedule item as .ics

Procurement — base: /api/procurement

Method	Path	Description
GET	/purchase-orders	List purchase orders
GET	/purchase-orders/export	Export purchase orders
POST	/purchase-orders	Create a purchase order
GET	/purchase-orders/:id	Get purchase order detail
PATCH	/purchase-orders/:id	Update purchase order
GET	/spare-parts	List spare-parts inventory
GET	/spare-parts/export	Export spare-parts inventory
POST	/spare-parts	Create spare-part record
GET	/spare-parts/:id	Get spare-part detail
PATCH	/spare-parts/:id	Update spare-part record

Method	Path	Description
POST	/spare-parts/:id/order	Raise a re-order for a spare part

Projects — base: /api/projects

Method	Path	Description
GET	/	List dynamic projects
POST	/	Create a project
PUT	/:id	Update a project
DELETE	/:id	Delete a project

Tasks — base: /api/tasks

Method	Path	Description
GET	/	List task-library entries
POST	/	Create a task
PUT	/:id	Update a task
DELETE	/:id	Delete a task

Checklists — base: /api/checklists

Method	Path	Description
GET	/	List checklist masters
GET	/equipment	List checklists applicable by equipment
GET	/task-master	List task-master entries usable in checklists
GET	/for-equipment	Resolve applicable checklist for a given equipment
POST	/job-cards/:id/apply	Apply a checklist template onto a job card
GET	/:id	Get checklist master detail
POST	/	Create checklist master
PUT	/:id	Update checklist master
DELETE	/:id	Delete checklist master

Audit — base: /api/audit

Method	Path	Description
GET	/	List audit log entries (filterable)
GET	/filters	Get available filter facets for the audit log
GET	/export	Export audit log (CSV/PDF,

Method	Path	Description
		recorded in export_audit)
GET	/:id	Get a single audit log entry with field-level changes

Appendix B: Database Table Inventory (New / Phase-3 Tables)

In addition to the pre-existing legacy "cmms_*" master tables (preserved and altered, not replaced), migrations 001 through 810 introduce the following tables:

Table(s)	Purpose
departments / sections	Organisational hierarchy (e.g. TIMCD department; T&ME, F&PE sections)
roles / permissions / role_permissions	RBAC core: 5 system roles, 40 atomic permissions, grant matrix
users / user_roles / user_role_history / user_lane_scopes	Login accounts, role assignment, change history, lane scoping
refresh_tokens / login_audit	JWT refresh-token persistence (hashed) and login attempt audit
employee_sso_directory / employee_sso_heads / sso_user_roles / sso_refresh_tokens	SSO authentication source and SSO-specific session tables
cmms_cont_mst	Vendor/contractor master, back-filled from legacy manufacturer values
equipment_status_history	Append-only equipment lifecycle transition log
job_request_status_history / job_request_accessories / job_request_terms	Job Request workflow history, accessory associations, and configurable T&Cs
job_card_status_history	Append-only Job Card lifecycle transition log
jc_calibration_equipment_used / jc_calibration_adjustments / jc_calibration_task_checklist / jc_observations_readings	Calibration job-card detail child tables
jc_maintenance_details	Maintenance job-card detail child table
jc_repair_equipment_used	Repair job-card detail child table
jc_task_checklist / jc_spares_used / jc_documents	Shared job-card child tables: applied checklist tasks, spares consumed, attached documents
task_library / checklists_master / checklist_master_tasks	Configurable checklist/task masters
purchase_orders / purchase_order_items / spare_parts	Procurement and inventory tables
schedules / schedule_status_history	Lab activity calendar and its status history
notifications	In-app notification feed
master_data_correction_requests	Master-data amendment request/approval workflow
operational_lanes	Definition of operational lanes (e.g. T&ME, F&PE) used for row-level scoping
audit_log / audit_log_changes / export_audit	System-wide audit trail and export-event log
schema_migrations	Migration-runner bookkeeping (checksum-gated

Table(s)	Purpose
	idempotency)

Appendix C: Role Summary

Role Code	Role Name	Description
SUPER_ADMIN	Super Admin	Master data + RBAC management + system integrity oversight (top tier admin)
LAB_IN_CHARGE	Lab In-Charge	Approve job requests, assign engineers, verify completed work, manage schedules
LAB_ENGINEER	Lab Engineer	Execute assigned jobs, fill job cards, record observations, generate PDFs
NORMAL_USER	Normal User	Raise job requests, register equipment, track own requests
VIEW_ONLY	View-Only User	Read all data; no write actions ever; auditor / management oversight

Roles are granted permissions through a role_permissions matrix seeded over a 40-entry permission catalogue (migrations 006 and 007); SUPER_ADMIN is granted every permission, while the remaining four roles receive a curated subset aligned to the descriptions above. Permission grants are additionally extended/tightened by later migrations as new capabilities were introduced (e.g., 400/410/420/430 series for reports/PDF/analytics/notifications permissions; 510 for schedule permissions; 600/610/620 for audit and bulk-verification permissions; 700/710 for lane-based RBAC scoping; 750 restricting equipment verification to Super Admin only; 806 for master-data-correction permissions).

Appendix D: Third-Party Library Bill of Materials

Backend (Node.js / Express)

Package	Version
express	^4.19.2
mysql2	^3.11.0
jsonwebtoken	^9.0.2
bcryptjs	^2.4.3
zod	^3.23.0
helmet	^7.1.0
cors	^2.8.5
compression	^1.7.4
cookie-parser	^1.4.7
express-rate-limit	^7.4.0
pino / pino-http / pino-pretty	^9.4.0 / ^10.3.0 / ^11.2.0
multer	^1.4.5-lts.1
pdfkit	^0.18.0
dayjs	^1.11.13
uuid	^9.0.1
dotenv	^16.4.5
envalid	^8.0.0
nodemon	^3.1.14

Frontend (React / Vite)

Package	Version
react / react-dom	^18.3.1
vite	^5.4.8
@vitejs/plugin-react	^4.3.2
react-router-dom	^6.27.0
@tanstack/react-query	^5.100.11
@tanstack/react-table	^8.21.3
axios	^1.7.7
zustand	^5.0.13
react-hook-form	^7.53.0
@hookform/resolvers	^3.9.0
zod	^3.23.0

Package	Version
dayjs	^1.11.20
recharts	^3.8.1
lucide-react	^0.454.0
clsx	^2.1.1
sonner	^2.0.7
tailwindcss	^3.4.13
@tailwindcss/forms	^0.5.9
postcss / autoprefixer	^8.4.47 / ^10.4.20